

## ✓ 1. Valid Parentheses (LeetCode 20)

**Problem:** Check if a string of `()[]{}`  is valid.

```
def isValid(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping.values():
            stack.append(char)
        elif char in mapping:
            if not stack or stack.pop() != mapping[char]:
                return False
    return not stack
```

## ✓ 2. Min Stack (LeetCode 155)

**Problem:** Design a stack that supports `push` , `pop` , `top` , and retrieving the minimum element.

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        min_val = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(min_val)

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]
```

## ✓ 3. Implement Stack using Queues (LeetCode 225)

**Problem:** Implement a stack using two queues.

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q = deque()

    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self):
        return self.q.popleft()

    def top(self):
        return self.q[0]

    def empty(self):
        return not self.q
```

## ✓ 4. Remove All Adjacent Duplicates (LeetCode 1047)

**Problem:** Remove adjacent duplicates in a string.

```
def removeDuplicates(s):
    stack = []
    for char in s:
        if stack and stack[-1] == char:
            stack.pop()
        else:
            stack.append(char)
    return ''.join(stack)
```

## ✓ 5. Baseball Game (LeetCode 682)

**Problem:** Simulate baseball game scoring using a stack.

```
def calPoints(ops):
    stack = []
    for op in ops:
```

```

    if op == '+':
        stack.append(stack[-1] + stack[-2])
    elif op == 'D':
        stack.append(2 * stack[-1])
    elif op == 'C':
        stack.pop()
    else:
        stack.append(int(op))
return sum(stack)

```

## ✓ 6. Final Prices With Discount (LeetCode 1475)

**Problem:** For each item, find the next item's price that is less or equal, and subtract it.

```

def finalPrices(prices):
    stack = []
    for i in range(len(prices)):
        while stack and prices[stack[-1]] >= prices[i]:
            j = stack.pop()
            prices[j] -= prices[i]
        stack.append(i)
    return prices

```

## ✓ 7. Next Greater Element I (LeetCode 496)

**Problem:** Find the next greater element for each element in nums1 from nums2.

```

def nextGreaterElement(nums1, nums2):
    stack, hashmap = [], {}
    for num in nums2:
        while stack and stack[-1] < num:
            hashmap[stack.pop()] = num
        stack.append(num)
    return [hashmap.get(num, -1) for num in nums1]

```

## ✓ 8. Backspace String Compare (LeetCode 844)

**Problem:** Check if two strings are equal after backspace # .

```
def build(s):
    stack = []
    for c in s:
        if c != '#':
            stack.append(c)
        elif stack:
            stack.pop()
    return ''.join(stack)

def backspaceCompare(s, t):
    return build(s) == build(t)
```

## ✓ 9. Evaluate Reverse Polish Notation (LeetCode 150)

**Problem:** Evaluate an expression in reverse Polish notation.

```
def evalRPN(tokens):
    stack = []
    for token in tokens:
        if token not in "+-*/":
            stack.append(int(token))
        else:
            b, a = stack.pop(), stack.pop()
            if token == '+': stack.append(a + b)
            elif token == '-': stack.append(a - b)
            elif token == '*': stack.append(a * b)
            elif token == '/': stack.append(int(a / b))
    return stack[0]
```

## ✓ 10. Make The String Great (LeetCode 1544)

**Problem:** Remove adjacent characters if one is upper and other is lowercase of same letter.

```
def makeGood(s):
    stack = []
    for c in s:
        if stack and abs(ord(stack[-1]) - ord(c)) == 32:
            stack.pop()
        else:
            stack.append(c)
    return ''.join(stack)
```